DOCUMENT EXTRACTION AND RECONSTRUCTION FOR MULTILINGUAL MACHINE TRANSLATION

Research Group 5

Presented by--

Samia Parish Qureshi
Shalvi Rastogi
Shambhavi

PROBLEM STATEMENT

With the increasing use of digital documents across government, education, healthcare, and enterprises, multilingual document translation has become essential. However, most existing machine translation systems translate only plain text and fail to preserve the original document structure, resulting in the loss of headings, tables, images, text boxes, formatting, and reading order. This often requires extensive manual editing after translation. Therefore, there is a need for an intelligent, layout-aware document translation system that can accurately extract, translate, and reconstruct documents while preserving their original structure and formatting across multiple languages.

01

Existing machine translation systems primarily process plain text and fail to preserve complex document structures such as headings, tables, images, text boxes, and formatting.

02

Loss of layout and logical reading order during translation reduces document readability and requires extensive manual reformatting, making the process time-consuming and inefficient.

03

There is a need for an intelligent, layout-aware multilingual translation framework that can accurately extract, translate, and reconstruct documents while maintaining their original structure and visual consistency.

PROJECT OBJECTIVES

The primary objective is to bridge the gap between linguistic translation and document layout preservation. The project aims to develop a unified framework that translates complex documents without distorting their original visual structure, nested tables, or logical reading order.

01. PRESERVE STRUCTURAL FIDELITY

To maintain strict structural consistency across complex file types (DOCX, PPTX, PDFs) by preserving essential elements like headings, text boxes, and nested tables during the translation phase.

02. ELIMINATE MANUAL REFORMATTING

To create a framework that dynamically handles challenges like text-length expansion in constrained regions, drastically reducing the manual formatting effort typically required post-translation.

03. ENHANCE MULTILINGUAL ACCESSIBILITY

To automate the translation of official records and complex reports, significantly streamlining workflows for e-governance and localization teams without distorting original layouts.

PROPOSED METHODOLOGY

Document Upload

- User uploads a DOCX, PDF, PPTX, or RTF file.
- Selects source and target languages via the React-based web interface.

Structure Extraction

- Backend extracts text, tables, headings, images, and layout using PyMuPDF, python-docx, python-pptx, and PaddleOCR.
- Preserves reading order and document formatting.

Intermediate JSON Representation

- Document is converted into a structured JSON containing:
  a. Text content
  b. Formatting attributes
  c. Spatial coordinates
  d. Layout metadata
- Ensures document structure remains intact during translation.

PROPOSED METHODOLOGY

Translation & Redis Caching

- Extracted text is translated using IndicTrans2 / Meta NLLB APIs.
- Redis checks for previously translated text:
  a. Cache Hit: Returns stored translation instantly.
  b. Cache Miss: Calls the translation model, stores the result, and returns the translation.

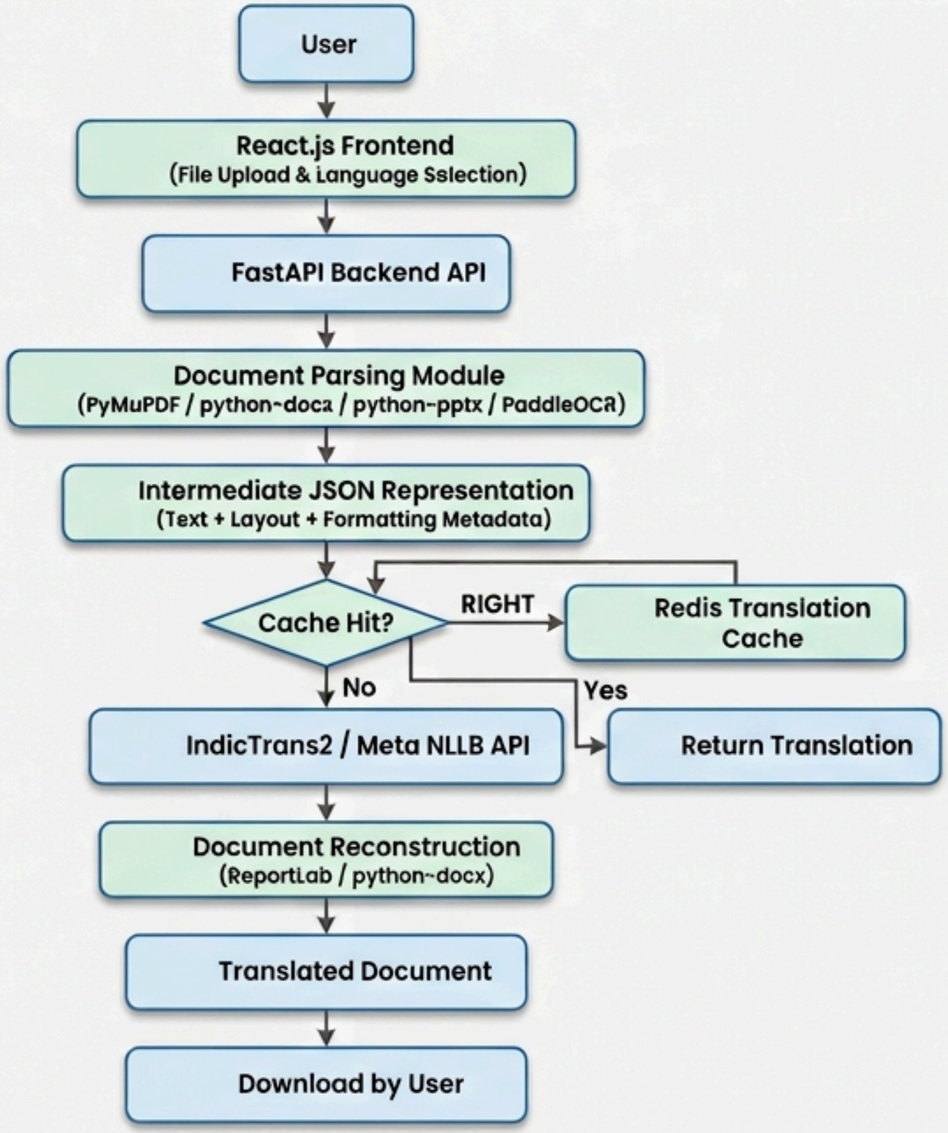
Document Reconstruction

- Translated text is merged back with the original layout using ReportLab and python-docx.
- Preserves formatting, tables, images, and document structure.

Output Generation

- User downloads the translated document in the same file format as the original.
- Ensures high translation accuracy while maintaining layout consistency.

TECHNICAL DESIGN – IMPLEMENTATION PIPELINE



MODULE WISE ARCHITECTURE

Module	Responsibilities
User Interface (Frontend)	Provides an intuitive interface for document upload, language selection, translation progress tracking, and downloading the translated document.
API & Workflow Manager	Provides an API nur workflow for document, translation, and translation.
Document Analysis & Extraction	Provides an parsing analysiss (PyMuPDF, python-docx, python-pptx, PaddlleOCR), and repeatnsnd ferontdaixes).
Document Structure Representation	Provides an document structure for text and Layout representation.
Translation Engine	Provides an translation inrace translation, translatied document.
Translation Memory & Caching	Provides an translation memory compteting & translated document.
Document Reconstruction Engine	Provides an document moduls, document Reconstruction, (ReportLab / python-docx).
Output & Delivery Module	Provides delivenry upload and downloading the translated document.

TECHNOLOGY STACK

- **Frontend:** React.js, Tailwind CSS, JavaScript, Axios.
- **Backend & Task Management**: Python (FastAPI), Redis.
- **Document AI & Parsing:** python-docx, python-pptx, PyMuPDF, PaddleOCR.
- **Structured Data Format:** JSON.
- **Document Reconstruction:** ReportLab.
- **AI Frameworks & Models:** IndicTrans2, Meta NLLB-200.

DATASETS AND RESOURCES

- M3T Dataset
- DocLayNet

WORK BREAKDOWN

Member 1:

- Develop React.js frontend
- Build FastAPI backend to receive uploaded files
- Detect document type (DOCX/PPTX/PDF) and native vs scanned PDFs
- Connect the frontend to the backend so users can track their upload progress

Member 2:

- Extract text from uploaded documents
- Extract layout and formatting information
- Convert extracted content into structured JSON

Member 3:

- Integrate IndicTrans2 API for multilingual translation
- Translate extracted JSON text with Redis caching for repeated segments
- Reconstruct the translated document, preserving original formatting and layout

EXECUTION PLAN

Month 1

Frontend & Setup

Develop the React.js interface and establish the basic FastAPI backend. Create the UI for file upload and target language selection.

1

Month 2

Parsing & Structuring

Implement PyMuPDF and python-docx for native document extraction. Convert extracted layouts and text into the intermediate JSON blueprint.

2

Month 3

API Integration

Integrate IndicTrans2 for translation, managed asynchronously via FastAPI's BackgroundTasks, with Redis caching for repeated segments.

3

Month 4

Document Reconstruction

Build the reconstruction module using ReportLab and python-docx. Map the translated text back to the original JSON spatial coordinates.

4

Month 5

OCR & Enhancement

Extend system capabilities to scanned PDFs by integrating PaddleOCR. Develop adaptive handling for text expansion in constrained tables.

5

Month 6

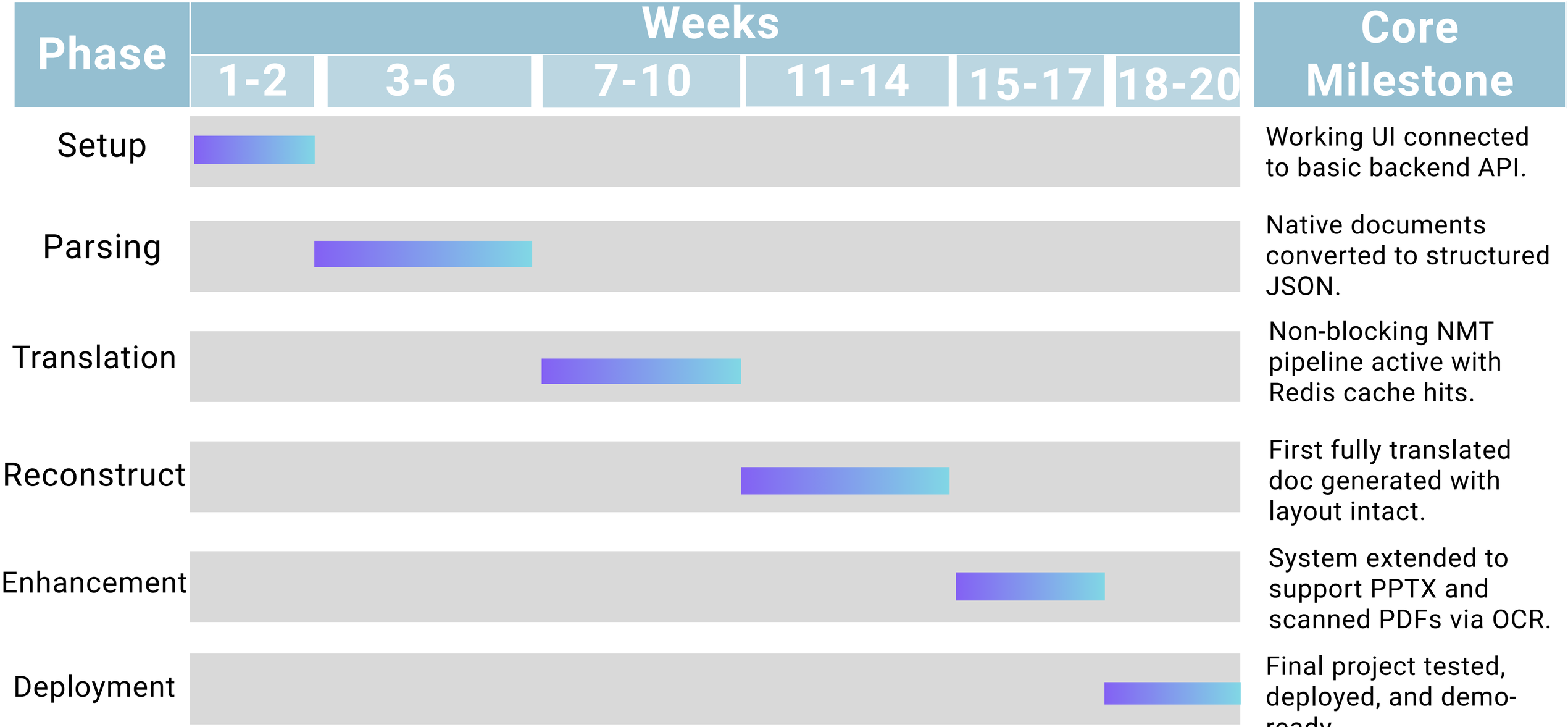
Testing & Deployment

Conduct end-to-end pipeline testing, perform layout accuracy benchmarking, and finalize the complete project deployment.

6

PROCESS TIMELINE

Gantt Chart



EXPECTED OUTCOMES

01.

Smart Layout Preservation: Maintains the original document structure including headings, tables, images, fonts, and formatting while translating the content.

02.

Faster & Automated Workflow: Automates the entire process from document upload to translated output, significantly reducing manual effort and processing time.

03.

Seamless Multilingual Translation: Produces accurate translations in the selected language while preserving the document's readability and professional appearance.

04.

Real-World Impact: Supports multilingual communication in domains such as e-governance, education, healthcare, and business by simplifying document translation and reconstruction.

THANK YOU